

NON FUNCTIONAL : ARSMF

① AVAILABILITY : % of time , service is accessible

100% avail = responds all the time

$$A(\text{in}\%) = \frac{(\text{Total time} - \text{Amount of down time})}{\text{Total time}} \times 100$$

- Availability measurement varies across providers:
- Some start from the service's launch.
- Others begin from a specific client's usage.
- Reported availability may exclude:
- Downtime affecting only some clients.
- Planned maintenance periods.
- Downtime caused by cyberattacks.
- Important to analyze how each provider calculates availability metrics.

② RELIABILITY (R)

- Probability that service will perform functions for a specified time
- R measures how service performs under varying operating conditions

MTBF: M time b/w Failures

MTTR : M time to Repair

$$MTBF = \frac{\text{Total time elapsed} - \text{sum of down time}}{\text{total num of Failures}}$$

$$MTTR = \frac{\text{Total maintenance time}}{\text{total num of repairs}}$$

Availability Percentages versus Service Downtime

Availability %	Downtime per Year	Downtime per Month	Downtime per Week
90% (1 nine)	36.5 days	72 hours	16.8 hours
99% (2 nines)	3.65 days	7.20 hours	1.68 hours
99.5% (2 nines)	1.83 days	3.60 hours	50.4 minutes
99.9% (3 nines)	8.76 hours	43.8 minutes	10.1 minutes
99.99% (4 nines)	52.56 minutes	4.32 minutes	1.01 minutes
99.999% (5 nines)	5.26 minutes	25.9 seconds	6.05 seconds
99.9999% (6 nines)	31.5 seconds	2.59 seconds	0.605 seconds
99.99999% (7 nines)	3.15 seconds	0.259 seconds	0.0605 seconds

- SOL : service level obj

- Healthy device — avail
 — reliable

- Measurement of A: time loss

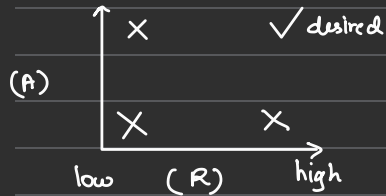
- measure of R: freq & impact of failure

- A & R are related

- A is a function of R

⇒ R is independent

A depends on R



③ SCALABILITY: — vertical
 — horizontal

- ✓ Increase workload same performance

- no. of req., Data storage

- ★ SIZE: new users to share DS

- ★ ADMINISTRATIVE: more orgs share DS

- ★ GEOGRAPHIC: new region

- ★ VERTICAL: scale up! more CPU, RAM.

expand current hardware & software

\$\$\$ → high

- ★ HORIZONTAL: scale-out! no. of machines

many nodes work as ONE.

④ MAINTAINABILITY :

After building system:

- find, fix BUGS
- Add features
- Updated platform

* OPERABILITY: smooth operation, no faults

* LUCIDITY: simple code. Understand & maintain

* MODIFIABILITY: new features

Measuring M :

- M is the probability that service will restore to its functions within a specified time of fault occurrence.
- regains normal operating conditions.

$$MTTR = \frac{\text{Total maintenance time}}{\text{Total number of repairs}}$$

(, mean time to repair

⑤ FAULT TOLERANCE

Replication
check-pointing.

- Large scale systems - 100s of DB, servers
billions of user requests

• Data safety & no recalculation of CPU intensive tasks
* FT = system's ability to execute persistently
even when 1 component fails.

- component $\rightarrow$ hw or sw

• REPLICATION: swap failed nodes w healthy ones
multiple copies : update regularly (challenge)
reduces availability.

Asynch updates: eventual consistency. } trade off
avoid vs consistency

CHECKPOINT • saves system state in stable storage.

- previously functioning check point roll back.

• consider 3 processes: i, j & k
2 msgs are exchanged.

one snapshot / checkpoint: C_{1i} , C_{1j} , C_{1k}

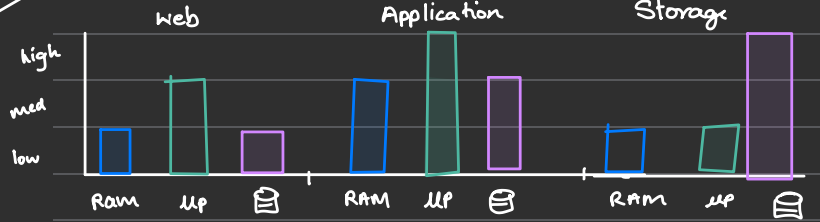
BOTEC: Back of the envelope Numbers

- Quick preliminary calculations (BOTECs)
- Diff nodes - Load balancers, App server, cache, DB
- Monolithic, μ service, modular etc.

Some eg: The number of:

- concurrent TCP conn server can support.

- Req per sec web, db or cache
- storage requirements: SERVERS



Web servers: First poc after LB.

Handles API calls from clients.

FB: 32 GB RAM 500 GB storage

App: core SW/ business logic. Dynamic content.
extensive μ P and storage.

RAM: 256 GB : rotating disks & flash 6.5TB

Storage servers: YT:

- Blob: encoded videos
- Temp μ P queue: 100 hours of video uploaded
- Bigtable: thumbnails of videos
- RDBMS: metadata comments, likes, views, etc.

Typical server:

μ P	Intel Xeon
cores	64
RAM	256
Cache L3	112.5MB
Storage capacity	16TB

Querying

QPS MySQL $\rightarrow$ 1000

key-value $\rightarrow$ 10k.

Cache server $\rightarrow$ 100k - 1 mil

REQUEST TYPES:

* Typical server types

Component	Count	holocene	
			nanosec
LLP	Intel Xenon	L1	0.9
Num of cores	64 cores	L2	3
RAM	256 GB	L3	13
Cache L3	112.5 MB	main memory	100
Storage capacity	16 TB	2 KB	3 μ s
		1 MB	9 μ s
		1 MB from SSD	200 μ s
msg/L	1000	Rountrip within data center	500 μ s
key-value	10,000	1 MB SSD I/O/s	1 milisec
Cache size	100,000-1 million	Disk seek	4 milisec
		2 MB seq	2 milli
		SF $\rightarrow$ NYC	7 mili

* REQUEST TYPES:

- CPU Bound: processor & node. 1 KB data + strip.
- memory bound: Read 1 MB data sequentially.
- IO bound: read 1 MB data sequentially from Disk.

$$\text{CPU time per program} = \text{Instruction per program} \times \text{CPI} \times \text{CPU time per clock cycle}$$

$$\text{eg: clock per sec for CPU} \sim 3.56 \text{ Hz clock rate} := 3.5 \times 10^9$$

$$\text{CPU time per clock cycle} = \frac{1}{3.5 \times 10^9}$$

per pgm:

$$C_{tpp} = \frac{1}{3.5 \times 10^9} \times 3.5 \times 10^6 = 0.001 \text{ second}$$

6.

BUILDING BLOCKS

• Bottom up

- 1) DNS: Distributed naming system
- 2) Load Balancers: incoming req → pool of servers
- 3) Database: CURD
- 4) Key-value pairs
- 5) CDN: viral content: video, image, webpage etc

The bottom-up approach for modern system design#

System design problems usually have some similarities, though specific details are often unique. We have extracted these similarities across design problems as the basic building blocks we'll be covering. One example of a building block is a load-balancing component, which we'll probably use in every design problem in one way or the other.

The purpose of separating the building blocks is to thoroughly discuss their design just once. This means that later we can use them anywhere without having to go over them in detail again. We can think about building blocks as bricks to construct more effective, capable systems.

Many of the building blocks we discuss are also available for actual use in the public clouds, such as Amazon Web Services (AWS), Azure, and Google Cloud Platform (GCP). We can use such constructs to build a system to further cement our understanding. (We won't construct the system in this course, but we've left it as an exercise for interested learners.)

6) Sequence: unique ID generator

7) Caching distributed

8) Messaging queue

9) Service monitoring

10) Publish-subscribe system

We'll discuss the following building blocks in detail:

Domain Name System: This building block focuses on how to design hierarchical and distributed naming systems for computers connected to the Internet via different Internet protocols.

Load Balancers: Here, we'll understand the design of a load balancer, which is used to fairly distribute incoming clients' requests among a pool of available servers. It also reduces load and can bypass failed servers.

Databases: This building block enables us to store, retrieve, modify, and delete data in connection with different data-processing procedures. Here, we'll discuss database types, replication, partitioning, and analysis of distributed databases.

Key-Value Store: It is a non-relational database that stores data in the form of a key-value pair. Here, we'll explain the design of a key-value store along with important concepts such as achieving scalability, durability, and configurability.



Content Delivery Network: In this chapter, we'll design a content delivery network (CDN) that's used to keep viral content such as videos, images, audio, and webpages. It efficiently delivers content to end users while reducing latency and burden on the data centers.



Sequencer: In this building block, we'll focus on the design of a unique IDs generator with a major focus on maintaining causality. It also explains three different methods for generating unique IDs.



Service Monitoring: Monitoring systems are critical in distributed systems because they help analyze the system and alert the stakeholders if a problem occurs. Monitoring is often useful to get early warning systems so that system administrators can act ahead of an impending problem becoming a huge issue. Here, we'll build two monitoring systems, one for the server-side and the other for client-side errors.

Distributed Caching: In this building block, we'll design a distributed caching system where multiple cache servers coordinate to store frequently accessed data.

Distributed Messaging Queue: In this building block, we'll focus on the design of a queue consisting of multiple servers, which is used between interacting entities called producers and consumers. It helps decouple producers and consumers, results in independent scalability, and enhances reliability.

Publish-Subscribe System: In this building block, we'll focus on the design of an asynchronous service-to-service communication method called a pub-sub system. It is popular in serverless, microservices architectures and data processing systems.

Rate Limiter: Here, we'll design a system that throttles incoming requests for a service based on the predefined limit. It is generally used as a defensive layer for services to avoid their excessive usage—whether intended or unintended.

Blob Store: This building block focuses on a storage solution for unstructured data—for example, multimedia files and binary executables.

Distributed Search: A search system takes a query from a user and returns relevant content in a few seconds or less. This building block focuses on the three integral components: crawl, index, and search.

Distributed Logging: Logging is an I/O intensive operation that is time-consuming and slow. Here, we'll design a system that allows services in a distributed system to log their events efficiently. The system will be made scalable and reliable.

Distributed Task Scheduling: We'll design a distributed task scheduler system that mediates between tasks and resources. It intelligently allocates resources to tasks to meet task-level and system-level goals. It's often used to offload background processing to be completed asynchronously.

Sharded Counters: This building block demonstrates an efficient distributed counting system to deal with millions of concurrent read/write requests, such as likes on a celebrity's tweet

Conventions#

For elaboration, we'll use a "Requirements" section whenever we design a building block (and a design problem). The "Requirements" section will highlight the deliverables we expect from the developed design.

"Requirements" will have two sub-categories:

Functional requirements: These represent the features a user of the designed system will be able to use. For example, the system will allow a user to search for content using the search bar.

Non-functional requirements (NFRs): The non-functional requirements are criteria based on which the user of a system will consider the system usable. NFR may include requirements like high availability, low latency, scalability, and so on.

Let's start with our building blocks.